

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Обработка изображений в интеллектуальных системах»

Тема: «Обучение классификаторов средствами библиотеки PyTorch»

Выполнил:

Студент 4 курса

Группы ИИ-26

Заруцкий В.Я.

Проверила:

Андренко К.В.

Цель работы: Научиться конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения

Ход работы:

1. Выполнить конструирование своей модели СНС, обучить ее на выборке по заданию (использовать **torchvision.datasets**). Предпочтение отдавать как можно более простым архитектурам, базирующимся на базовых типах слоев (сверточный, полносвязный, подвыборочный, слой нелинейного преобразования). Оценить эффективность обучения на тестовой выборке, построить график изменения ошибки (matplotlib);
2. Ознакомьтесь с state-of-the-art результатами для предлагаемых выборок (из материалов в сети Интернет). Сделать выводы о результатах обучения СНС из п. 1;
3. Реализовать визуализацию работы СНС из пункта 1 (выбор и подачу на архитектуру произвольного изображения с выводом результата);
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

№ варианта	Выборка	Размер исходного изображения	Оптимизатор
4	CIFAR-100	32X32	SGD

Код программы:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader

import torchvision
import torchvision.transforms as transforms

import numpy as np
import matplotlib.pyplot as plt
import random
from PIL import Image
from collections import defaultdict

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Используемое устройство:", device)

MEAN = (0.5071, 0.4865, 0.4409)
STD = (0.2673, 0.2564, 0.2762)
```

```

transform_train = transforms.Compose([
    transforms.RandomCrop(32, padding=4),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(MEAN, STD),
])

transform_test = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(MEAN, STD),
])

train_set = torchvision.datasets.CIFAR100(root='./data', train=True,
                                          download=True, transform=transform_train)
test_set = torchvision.datasets.CIFAR100(root='./data', train=False,
                                          download=True, transform=transform_test)

CLASSES = train_set.classes          # список из 100 названий классов
NUM_CLASSES = len(CLASSES)

BATCH_SIZE = 128
train_loader = DataLoader(train_set, batch_size=BATCH_SIZE, shuffle=True, num_workers=2)
test_loader = DataLoader(test_set, batch_size=256, shuffle=False, num_workers=2)

print(f"Train: {len(train_set)}, Test: {len(test_set)}, "
      f"классов: {NUM_CLASSES}, размер изображения: {train_set[0][0].shape}")

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=NUM_CLASSES):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

        self.fc1 = nn.Linear(128 * 4 * 4, 256)
        self.fc2 = nn.Linear(256, num_classes)
        self.dropout = nn.Dropout(0.4)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x

model = SimpleCNN().to(device)
n_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(model)
print(f"Обучаемых параметров: {n_params:,}")

criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9, weight_decay=5e-4)
N_EPOCHS = 25

history = {"train_loss": [], "train_acc": [], "test_loss": [], "test_acc": []}

```

```

def run_epoch(loader, train=True):
    model.train() if train else model.eval()
    total_loss, correct, total = 0.0, 0, 0
    context = torch.enable_grad() if train else torch.no_grad()
    with context:
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            if train:
                optimizer.zero_grad()
                outputs = model(images)
                loss = criterion(outputs, labels)
                if train:
                    loss.backward()
                    optimizer.step()
                total_loss += loss.item() * images.size(0)
                correct += (outputs.argmax(1) == labels).sum().item()
                total += labels.size(0)
    return total_loss / total, correct / total

for epoch in range(1, N_EPOCHS + 1):
    train_loss, train_acc = run_epoch(train_loader, train=True)
    test_loss, test_acc = run_epoch(test_loader, train=False)
    history["train_loss"].append(train_loss)
    history["train_acc"].append(train_acc)
    history["test_loss"].append(test_loss)
    history["test_acc"].append(test_acc)
    print(f"Эпоха {epoch:2d}/{N_EPOCHS} | train loss {train_loss:.4f} acc {train_acc*100:5.2f}%
| "
        f"test loss {test_loss:.4f} acc {test_acc*100:5.2f}%")

epochs_range = range(1, N_EPOCHS + 1)
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

axes[0].plot(epochs_range, history["train_loss"], label="Train loss", marker='o', markersize=3)
axes[0].plot(epochs_range, history["test_loss"], label="Test loss", marker='o', markersize=3)
axes[0].set_xlabel("Эпоха"); axes[0].set_ylabel("Loss")
axes[0].set_title("Изменение ошибки при обучении"); axes[0].legend(); axes[0].grid(alpha=0.3)

axes[1].plot(epochs_range, [a*100 for a in history["train_acc"]], label="Train acc", marker='o',
markersize=3)
axes[1].plot(epochs_range, [a*100 for a in history["test_acc"]], label="Test acc", marker='o',
markersize=3)
axes[1].set_xlabel("Эпоха"); axes[1].set_ylabel("Accuracy, %")
axes[1].set_title("Изменение точности при обучении"); axes[1].legend(); axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()

print(f"Итоговая точность на тесте: {history['test_acc'][-1]*100:.2f}%")

# --- Точность по классам. Для CIFAR-100 классов 100, печатаем ТОП-10 лучших и худших ---
model.eval()
class_correct, class_total = defaultdict(int), defaultdict(int)

with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        preds = model(images).argmax(1)
        for label, pred in zip(labels, preds):
            class_total[label.item()] += 1

```

```

class_correct[label.item()] += int(label.item() == pred.item())

per_class_acc = {i: (100 * class_correct[i] / class_total[i]) for i in range(NUM_CLASSES)}
sorted_acc = sorted(per_class_acc.items(), key=lambda kv: kv[1])

print("Топ-10 ЛУЧШИХ классов:")
for i, acc in sorted_acc[-10:][::-1]:
    print(f" {CLASSES[i]:>15s}: {acc:5.2f}%")

print("Топ-10 ХУДШИХ классов:")
for i, acc in sorted_acc[:10]:
    print(f" {CLASSES[i]:>15s}: {acc:5.2f}%")

def _topk_bars(probs, k=10, highlight=None):
    """Возвращает (индексы, вероятности в %) для топ-k классов.
    highlight — индекс класса, который надо выделить красным."""
    idx = np.argsort(probs)[::-1][:k]
    labels = [CLASSES[i] for i in idx]
    vals = probs[idx] * 100
    colors = ['red' if (highlight is not None and i == highlight) else 'steelblue' for i in idx]
    return labels, vals, colors

def visualize_test_sample(index=None):
    model.eval()
    if index is None:
        index = random.randrange(len(test_set))
    image, true_label = test_set[index]
    input_tensor = image.unsqueeze(0).to(device)
    with torch.no_grad():
        probs = F.softmax(model(input_tensor), dim=1).cpu().numpy().flatten()
    pred_label = int(np.argmax(probs))

    def denorm(img):
        img = img.numpy().transpose(1, 2, 0) * np.array(STD) + np.array(MEAN)
        return np.clip(img, 0, 1)

    fig, axes = plt.subplots(1, 2, figsize=(11, 5), gridspec_kw={'width_ratios': [1, 1.6]})
    axes[0].imshow(denorm(image)); axes[0].axis('off')
    correct = pred_label == true_label
    axes[0].set_title(f"Истина: {CLASSES[true_label]}\nПредсказание: {CLASSES[pred_label]}",
                     color='green' if correct else 'red')

    labels, vals, colors = _topk_bars(probs, k=10, highlight=pred_label)
    axes[1].barh(labels, vals, color=colors)
    axes[1].set_xlabel("Вероятность, %")
    axes[1].set_xlim(0, max(100, vals.max() + 5))
    axes[1].invert_yaxis()
    axes[1].set_title("Топ-10 предсказаний")
    plt.tight_layout(); plt.show()

for _ in range(3):
    visualize_test_sample()

def predict_custom_image(path_or_pil):
    model.eval()
    img = Image.open(path_or_pil).convert('RGB') if isinstance(path_or_pil, str) else
    path_or_pil.convert('RGB')
    preprocess = transforms.Compose([
        transforms.Resize((32, 32)),
        transforms.ToTensor(),

```

```

        transforms.Normalize(MEAN, STD),
    ])
    input_tensor = preprocess(img).unsqueeze(0).to(device)
    with torch.no_grad():
        probs = F.softmax(model(input_tensor), dim=1).cpu().numpy().flatten()
    pred_label = int(np.argmax(probs))

    fig, axes = plt.subplots(1, 2, figsize=(11, 5), gridspec_kw={'width_ratios': [1, 1.6]})
    axes[0].imshow(img); axes[0].axis('off')
    axes[0].set_title(f"Предсказание сети: {CLASSES[pred_label]}")

    labels, vals, colors = _topk_bars(probs, k=10, highlight=pred_label)
    axes[1].barh(labels, vals, color=colors)
    axes[1].set_xlabel("Вероятность, %")
    axes[1].set_xlim(0, max(100, vals.max() + 5))
    axes[1].invert_yaxis()
    axes[1].set_title("Топ-10 предсказаний")
    plt.tight_layout(); plt.show()
    return pred_label, probs

predict_custom_image(r'1.jpg')

```

Результат тестирования:

Train: 50000, Test: 10000, классов: 100, размер изображения: torch.Size([3, 32, 32])

SimpleCNN(

```

    (conv1): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (conv3): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (fc1): Linear(in_features=2048, out_features=256, bias=True)
    (fc2): Linear(in_features=256, out_features=100, bias=True)
    (dropout): Dropout(p=0.4, inplace=False)
)

```

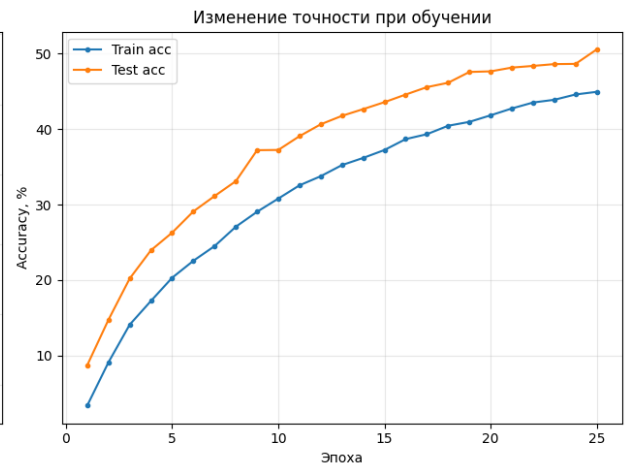
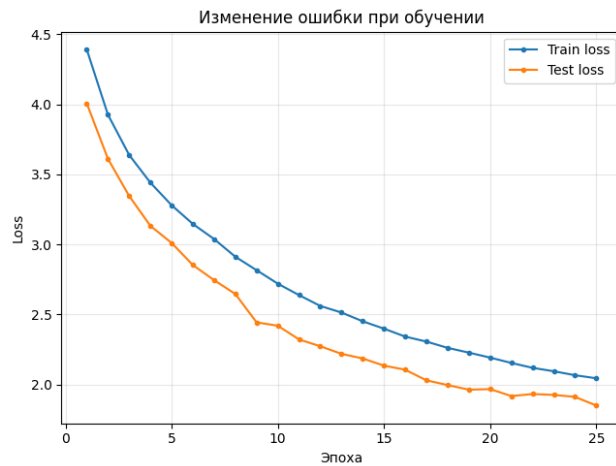
Обучаемых параметров: 643,492

```

Эпоха 1/25 | train loss 4.3920 acc 3.39% | test loss 4.0072 acc 8.73%
Эпоха 2/25 | train loss 3.9253 acc 9.11% | test loss 3.6107 acc 14.75%
Эпоха 3/25 | train loss 3.6380 acc 14.10% | test loss 3.3454 acc 20.21%
Эпоха 4/25 | train loss 3.4405 acc 17.24% | test loss 3.1322 acc 23.95%
Эпоха 5/25 | train loss 3.2794 acc 20.32% | test loss 3.0103 acc 26.29%
Эпоха 6/25 | train loss 3.1460 acc 22.56% | test loss 2.8527 acc 29.10%
Эпоха 7/25 | train loss 3.0381 acc 24.51% | test loss 2.7450 acc 31.14%
Эпоха 8/25 | train loss 2.9119 acc 27.08% | test loss 2.6462 acc 33.11%
Эпоха 9/25 | train loss 2.8160 acc 29.05% | test loss 2.4438 acc 37.21%
Эпоха 10/25 | train loss 2.7199 acc 30.80% | test loss 2.4198 acc 37.24%
Эпоха 11/25 | train loss 2.6382 acc 32.56% | test loss 2.3218 acc 39.07%
Эпоха 12/25 | train loss 2.5609 acc 33.77% | test loss 2.2732 acc 40.64%
Эпоха 13/25 | train loss 2.5138 acc 35.24% | test loss 2.2191 acc 41.76%
Эпоха 14/25 | train loss 2.4513 acc 36.19% | test loss 2.1862 acc 42.65%

```

Эпоха 15/25 | train loss 2.3987 acc 37.23% | test loss 2.1348 acc 43.57%
 Эпоха 16/25 | train loss 2.3421 acc 38.68% | test loss 2.1057 acc 44.58%
 Эпоха 17/25 | train loss 2.3066 acc 39.33% | test loss 2.0300 acc 45.56%
 Эпоха 18/25 | train loss 2.2617 acc 40.45% | test loss 1.9961 acc 46.14%
 Эпоха 19/25 | train loss 2.2277 acc 40.96% | test loss 1.9633 acc 47.57%
 Эпоха 20/25 | train loss 2.1922 acc 41.83% | test loss 1.9675 acc 47.65%
 Эпоха 21/25 | train loss 2.1541 acc 42.74% | test loss 1.9178 acc 48.15%
 Эпоха 22/25 | train loss 2.1191 acc 43.52% | test loss 1.9323 acc 48.37%
 Эпоха 23/25 | train loss 2.0947 acc 43.89% | test loss 1.9261 acc 48.61%
 Эпоха 24/25 | train loss 2.0665 acc 44.59% | test loss 1.9119 acc 48.64%
 Эпоха 25/25 | train loss 2.0447 acc 44.94% | test loss 1.8501 acc 50.56%



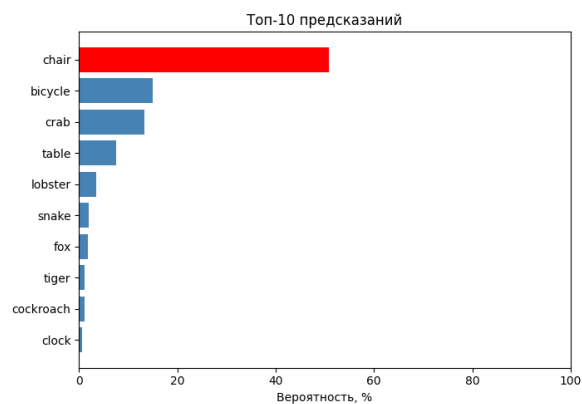
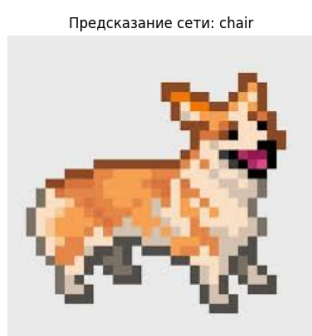
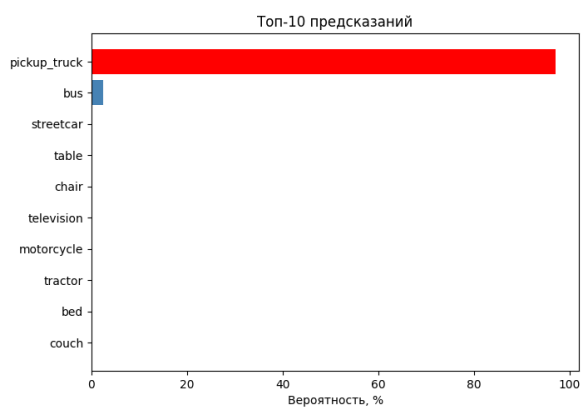
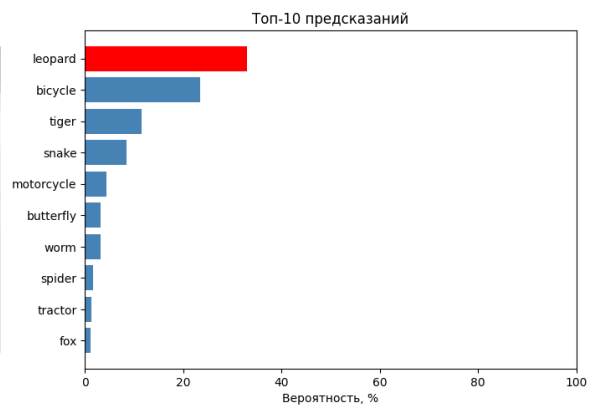
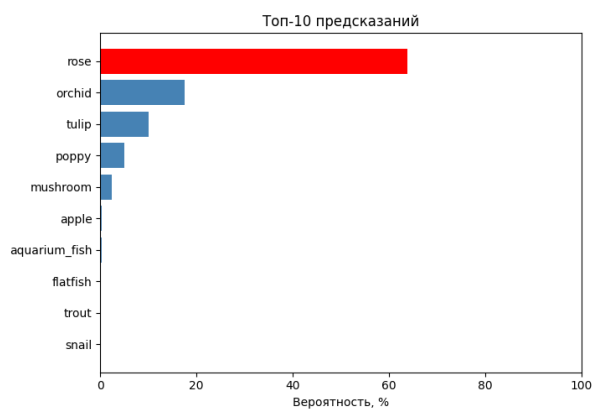
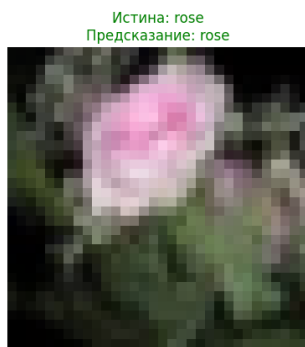
Итоговая точность на тесте: 50.56%

Топ-10 ЛУЧШИХ классов:

plain: 84.00%
 wardrobe: 83.00%
 road: 83.00%
 skunk: 82.00%
 motorcycle: 80.00%
 sea: 79.00%
 skyscraper: 75.00%
 orange: 75.00%
 chair: 74.00%
 apple: 74.00%

Топ-10 ХУДШИХ классов:

otter: 7.00%
 seal: 10.00%
 bear: 17.00%
 woman: 17.00%
 squirrel: 18.00%
 bus: 20.00%
 turtle: 20.00%
 possum: 24.00%
 clock: 25.00%
 mouse: 25.00%



Вывод: Прodelав данную лабораторную работу я научился конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения